

# Redes Neurais Artificiais



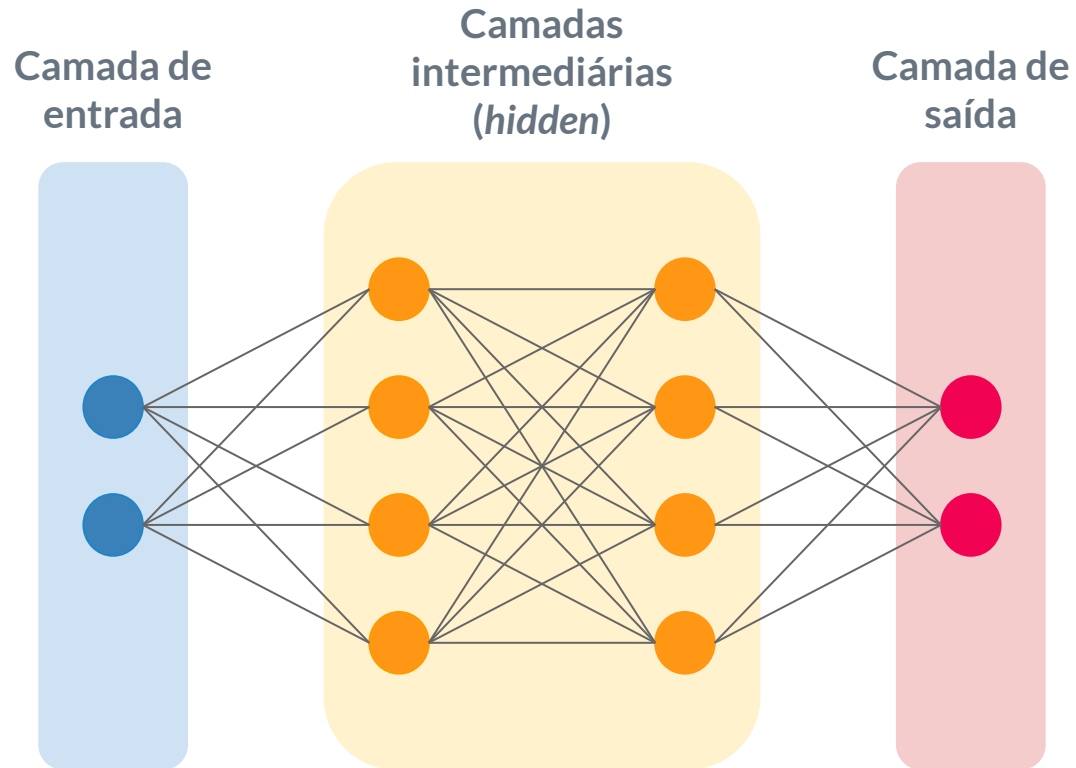
Gabriel P. Oliveira

Inteligência Artificial  
2026/1

# Redes Neurais



# O que são Redes Neurais?



# Por que usar Redes Neurais?

As Redes Neurais Artificiais (RNAs) são modelos de computação que têm as seguintes propriedades:

- ▷ Capacidade de **aprender** a partir de exemplos
- ▷ Capacidade de **generalização** (prever dados não vistos)
- ▷ Tolerância a falhas e ruídos

1.

# Componentes Básicos das Redes Neurais

*Quais são?*

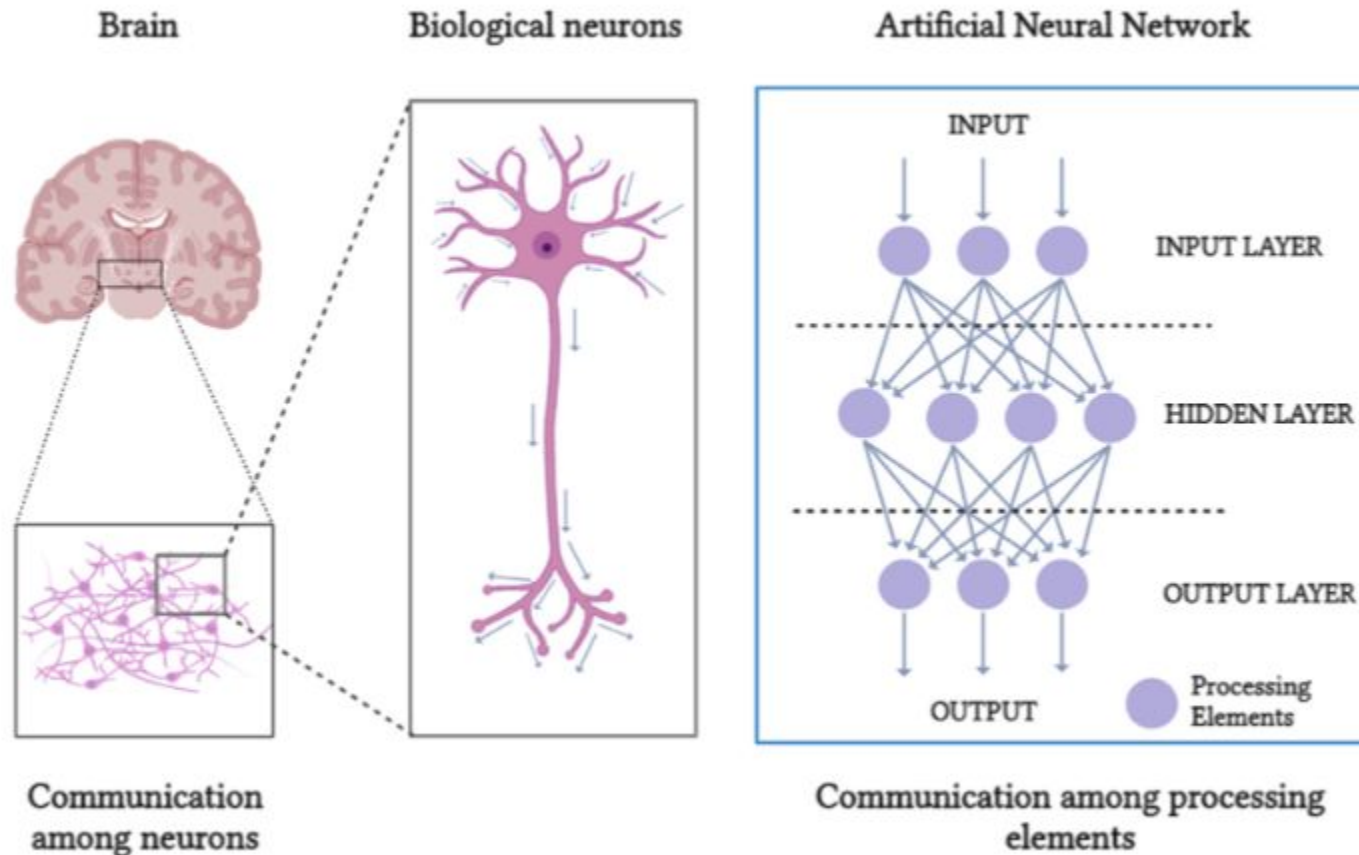
# Componentes das RNAs

As Redes Neurais Artificiais (RNAs) são sistemas compostos por unidades simples, conhecidas como **neurônios artificiais**

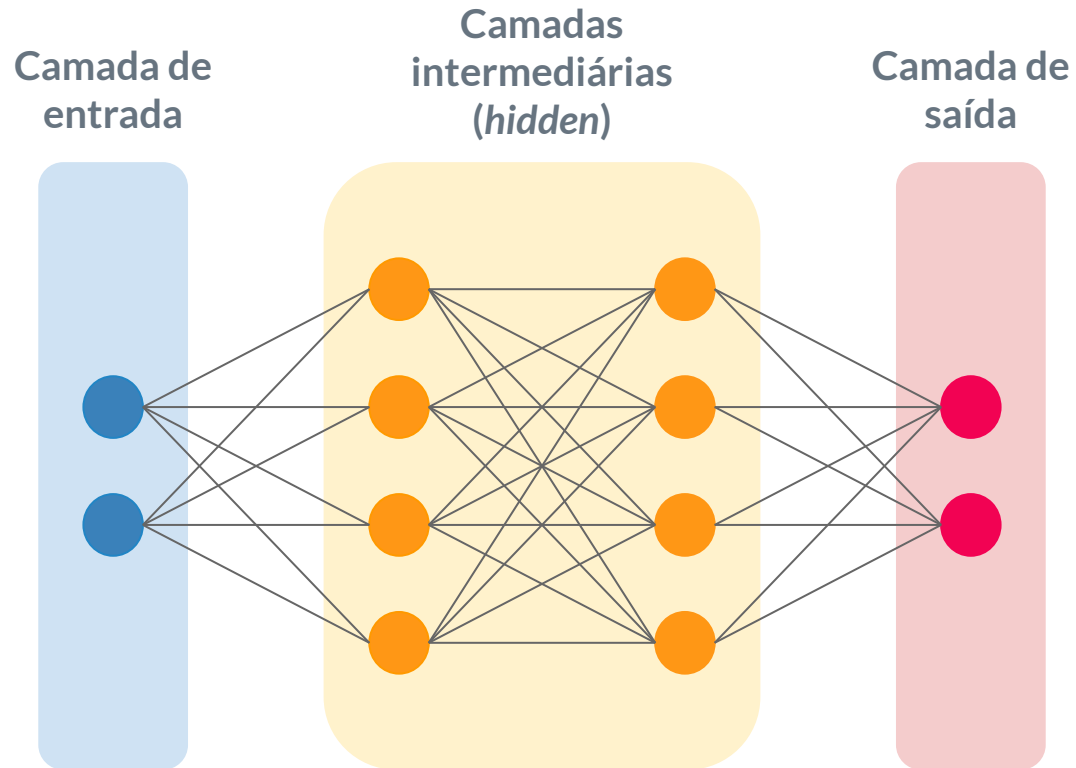
- ▷ Computam **funções matemáticas**
- ▷ Estão dispostas em **uma ou mais camadas**
- ▷ Interligadas por muitas **conexões**
  - Geralmente unidirecionais
  - Simulam as sinapses biológicas

# Inspiração das Redes Neurais

## Cérebro humano

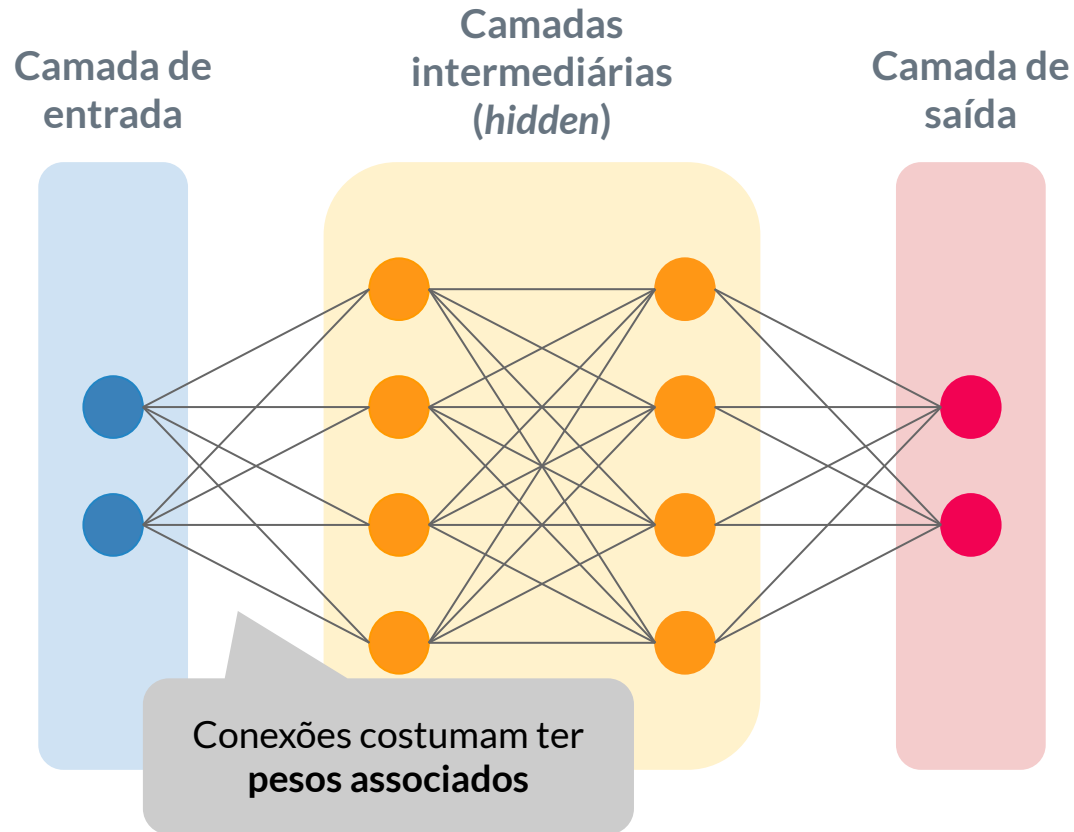


# O que são Redes Neurais?





# O que são Redes Neurais?



# Componentes das RNAs

## Pesos das conexões

- ▷ Podem assumir valores positivos ou negativos
  - Significa que o comportamento da conexão é *excitatório* ou *inibitório*
- ▷ Os valores são ajustados em um **processo de aprendizado** (treinamento)

# Componentes das RNAs

## Estrutura geral das RNAs:

- ▷ Unidades de processamento  $n_i$  (nodos)
- ▷ Conexões com pesos  $w_{ij}$
- ▷ Saída
- ▷ Topologia

# Componentes das RNAs

## Estrutura geral das RNAs:

- ▷ Unidades de processamento  $n_i$  (nodos)
- ▷ Conexões com pesos  $w_{ij}$
- ▷ Saída
- ▷ Topologia

**Objetivo** → ajustar os pesos da rede para minimizar alguma medida do erro no conjunto de treinamento

# Componentes das RNAs

## Estrutura geral das RNAs:

- ▷ Unidades de processamento  $n_i$  (nodos)
- ▷ Conexões com pesos  $w_{ij}$
- ▷ Saída
- ▷ Topologia

**Objetivo** → ajustar os pesos da rede para minimizar alguma medida do erro no conjunto de treinamento

A aprendizagem é formulada como uma **busca de otimização** no **espaço de pesos**

# Componentes das RNAs

## Aspectos básicos de uma RNA:

### *Arquitetura*

- Tipo e número de unidades de processamento
- Forma como os neurônios estão conectados

### *Aprendizado*

- Regras utilizadas para ajuste de pesos

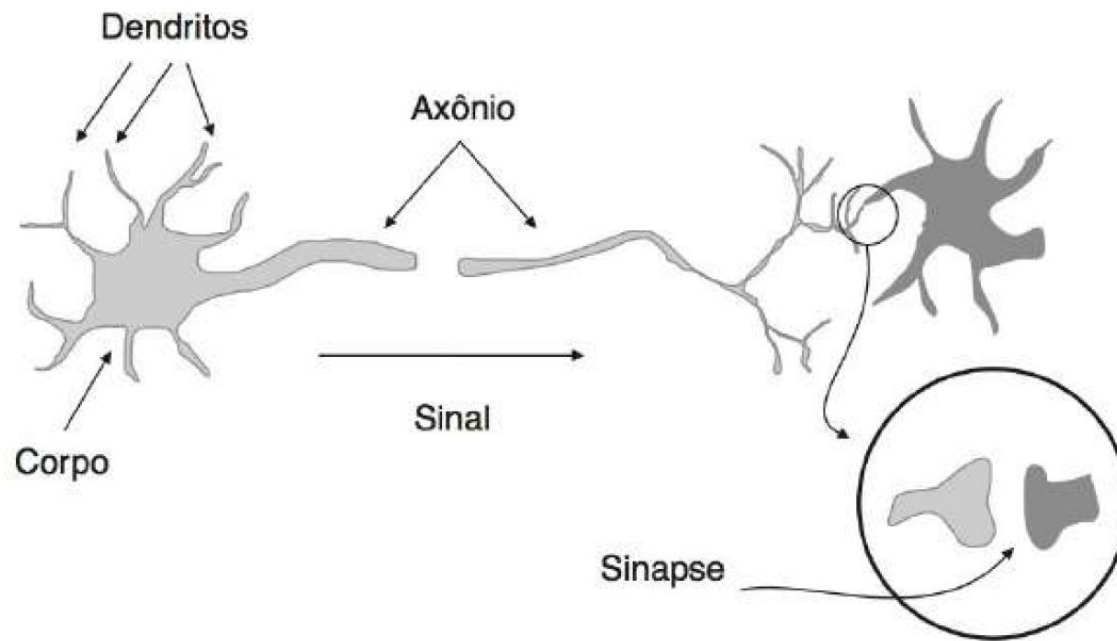
2.

# Arquitetura das RNAs

*Como funcionam?*

# Redes Biológicas

Neurônio → Unidade fundamental do sistema nervoso

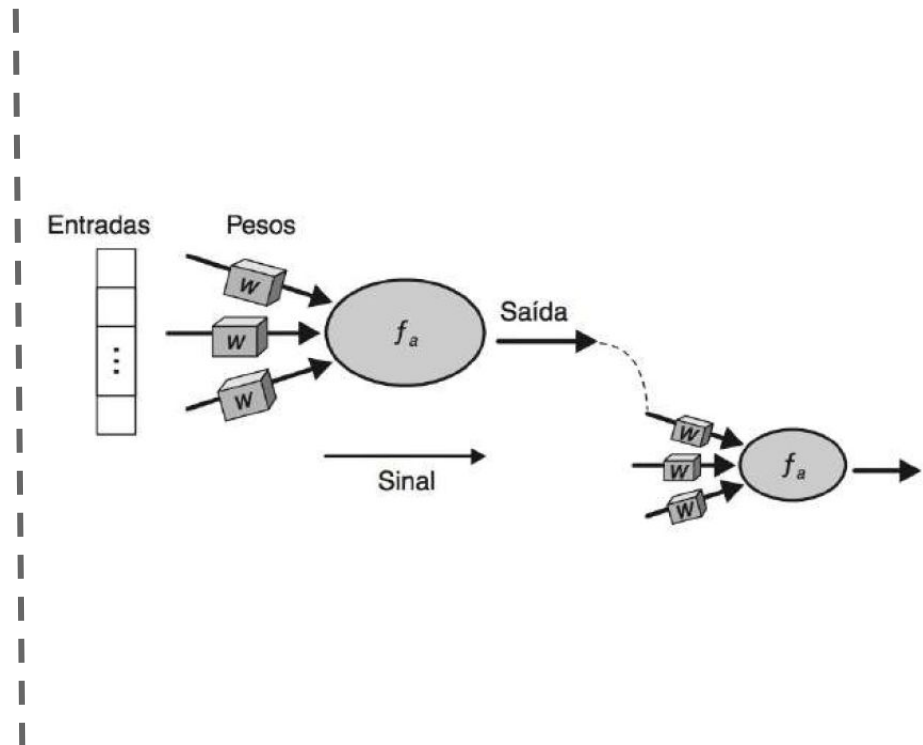
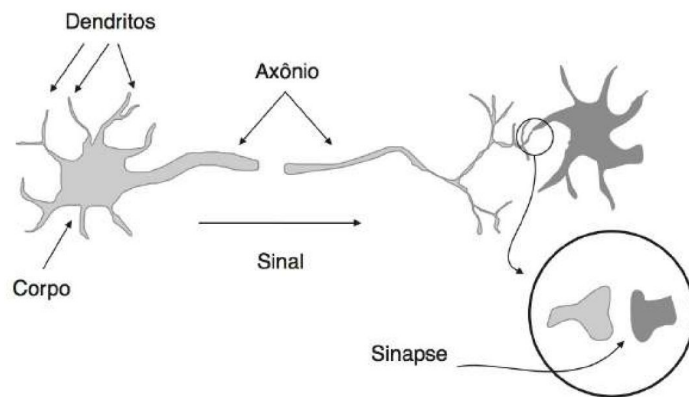


Neurônio biológico simplificado



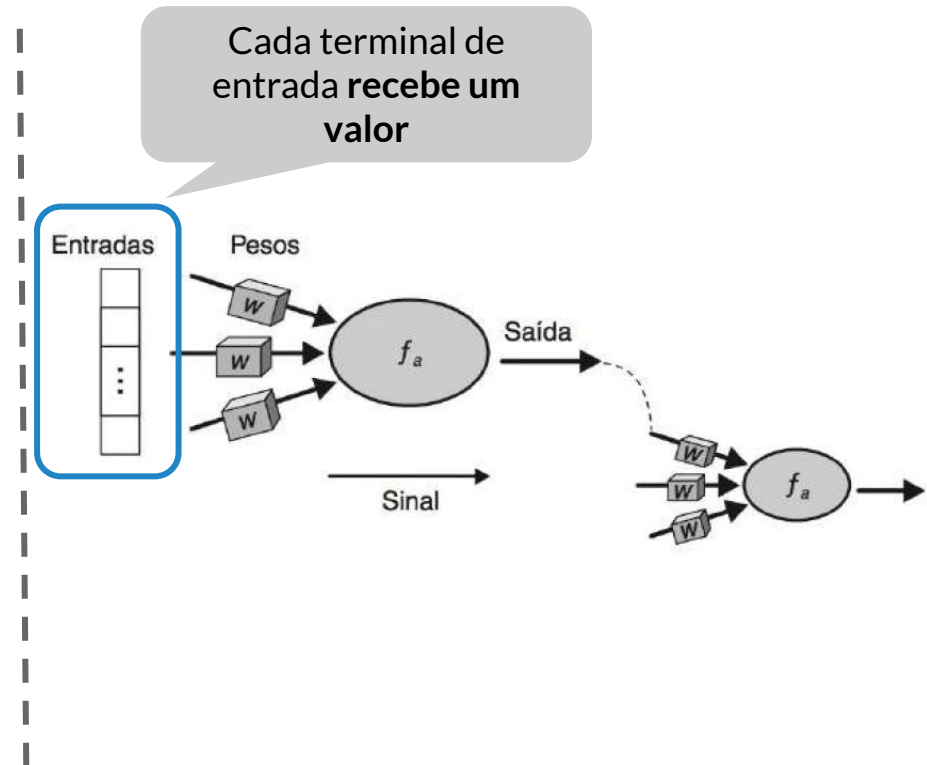
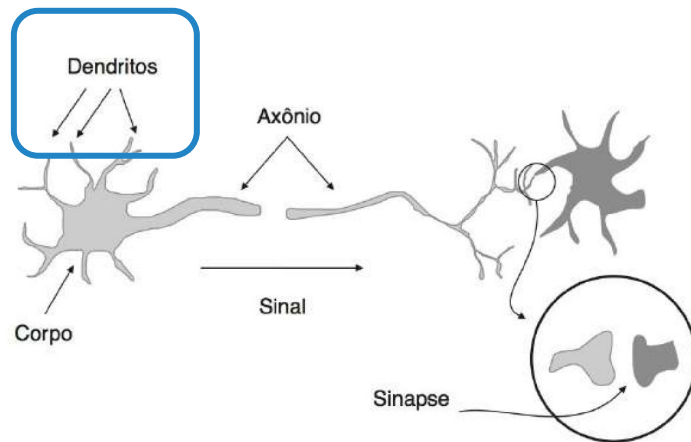
# Neurônio Artificial

Neurônio → Unidade fundamental de uma RNA



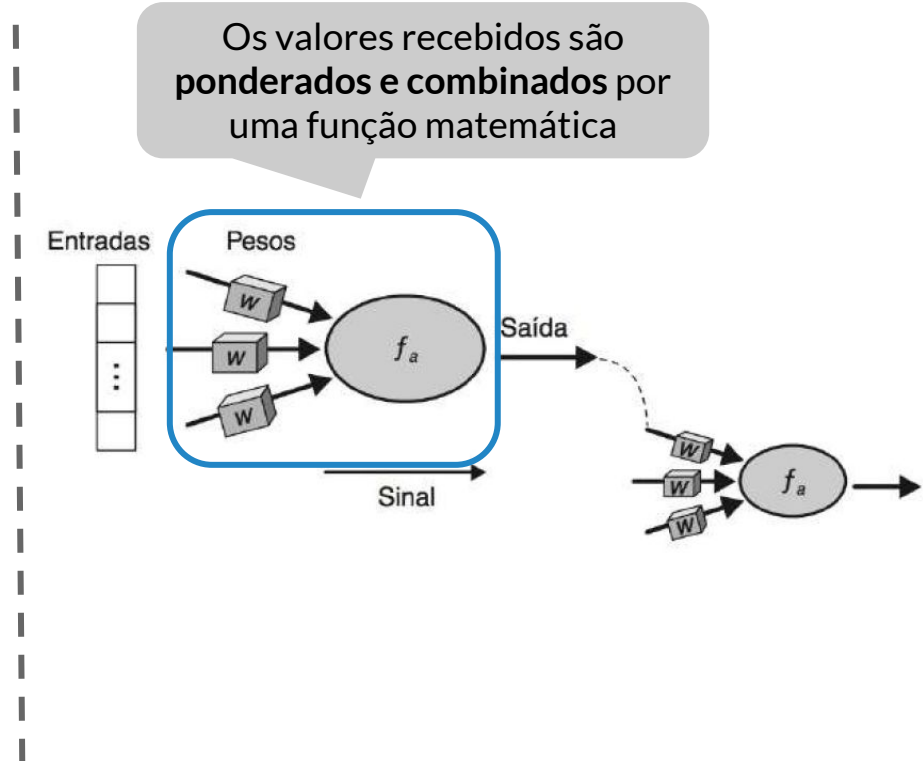
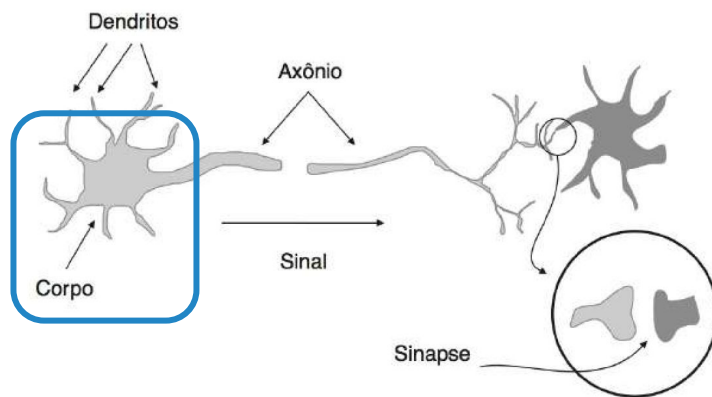
# Neurônio Artificial

Neurônio → Unidade fundamental de uma RNA



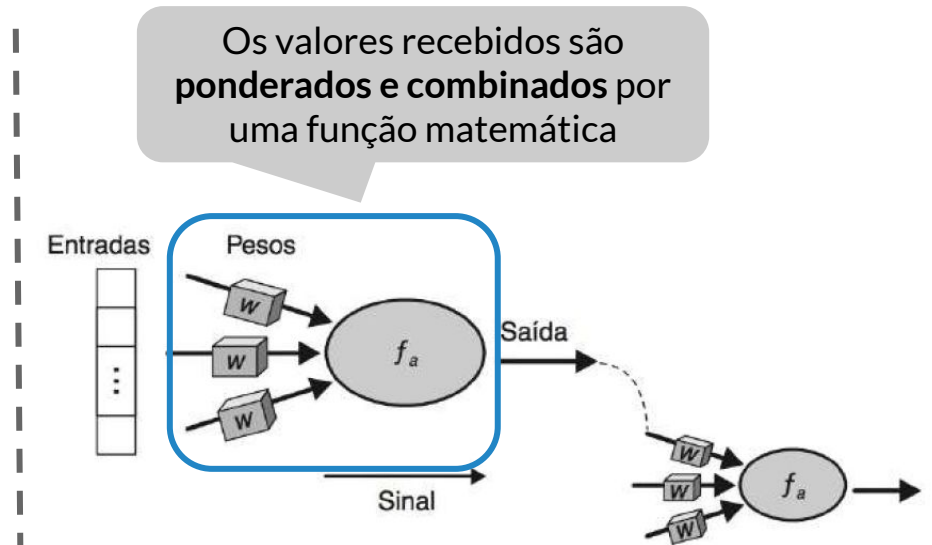
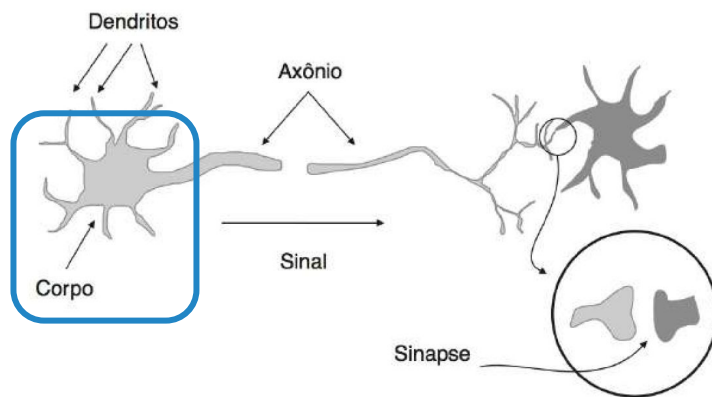
# Neurônio Artificial

Neurônio → Unidade fundamental de uma RNA



# Neurônio Artificial

Neurônio → Unidade fundamental de uma RNA



Em outras palavras:

1. Recebe entradas (valores numéricos)
2. Multiplica por pesos e soma tudo
3. Aplica uma **função de ativação** ao resultado

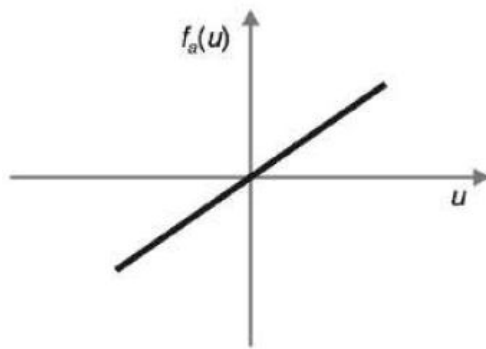
# Funções de Ativação

**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe

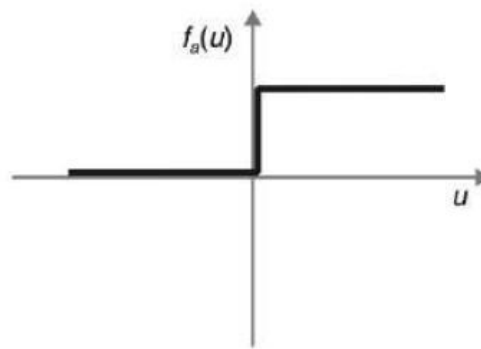
- ▷ Pode adicionar não-linearidade na rede, por exemplo

# Funções de Ativação

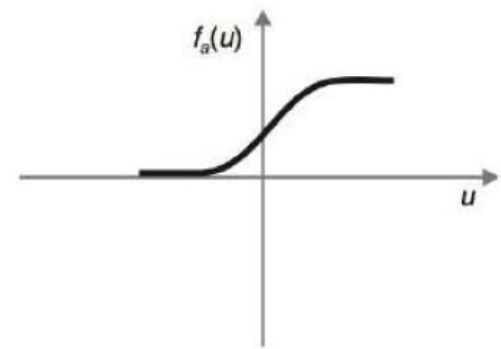
**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe



(a) Linear



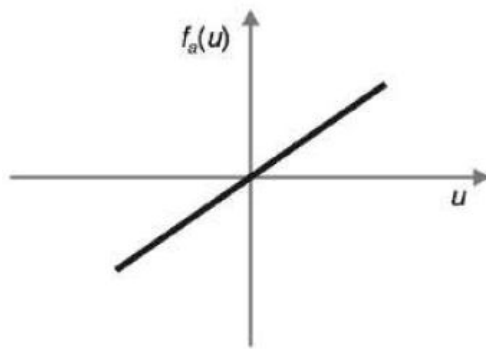
(b) Limiar



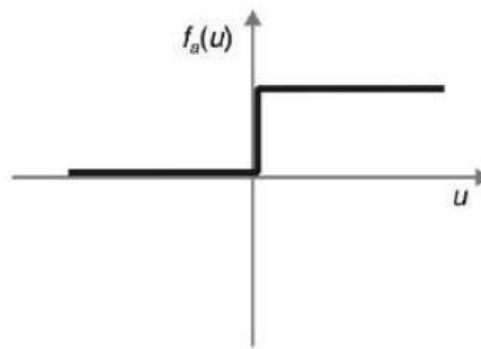
(c) Sigmoidal

# Funções de Ativação

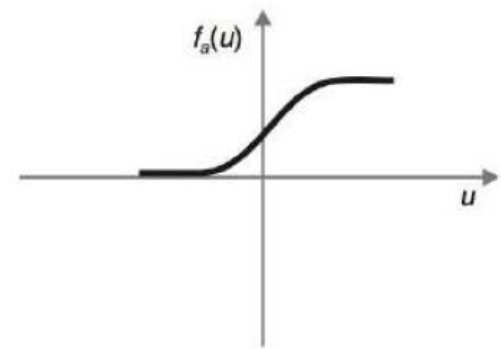
**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe



(a) Linear



(b) Limiar

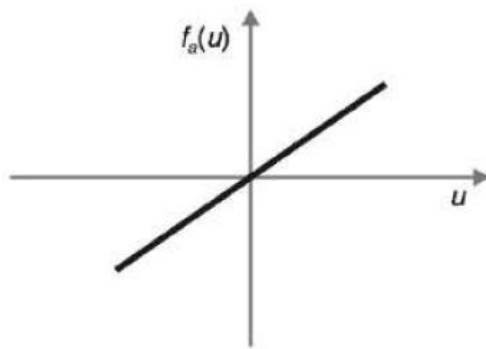


(c) Sigmoidal

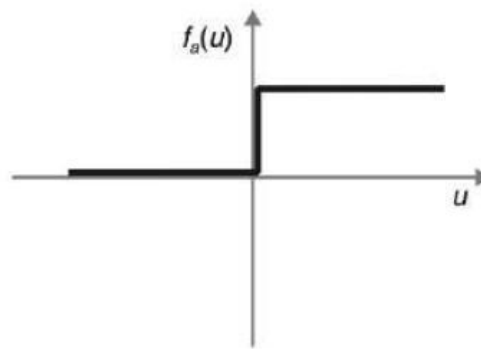
Retorna a **soma ponderada**  
dos valores de entrada

# Funções de Ativação

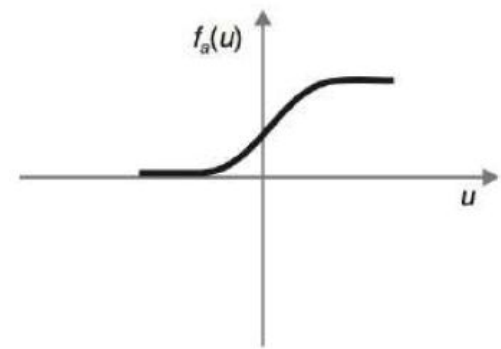
**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe



(a) Linear



(b) Limiar



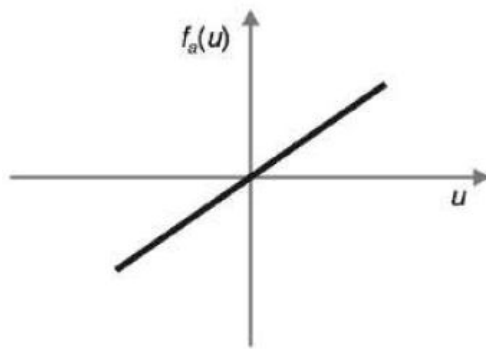
(c) Sigmoidal

O valor do limiar define quando o resultado será igual a 1 ou 0

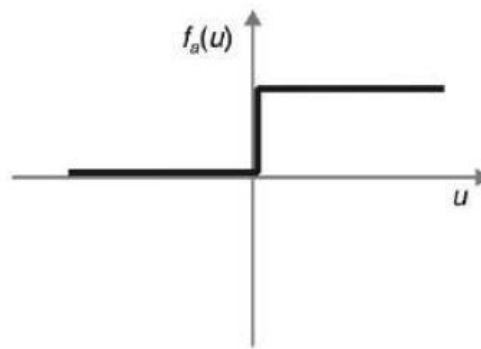


# Funções de Ativação

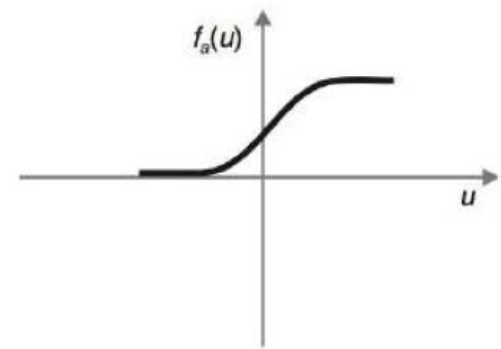
**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe



(a) Linear



(b) Limiar

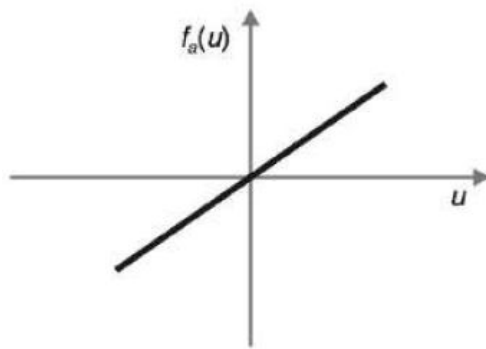


(c) Sigmoidal

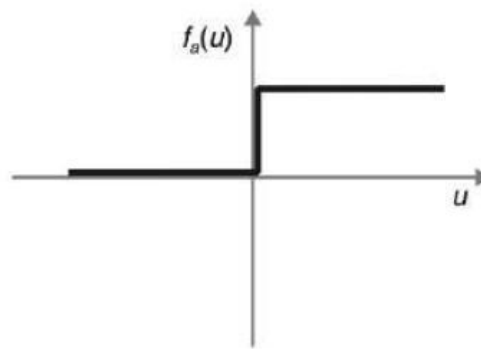
**Interruptor** → quando a soma ponderada das entradas ultrapassa o limiar, o neurônio fica ativo

# Funções de Ativação

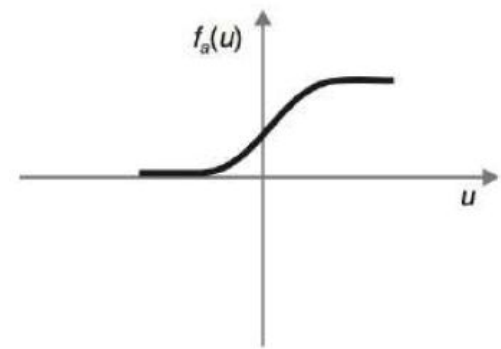
**Objetivo** → controlar o que cada neurônio “faz” com a informação que recebe



(a) Linear



(b) Limiar

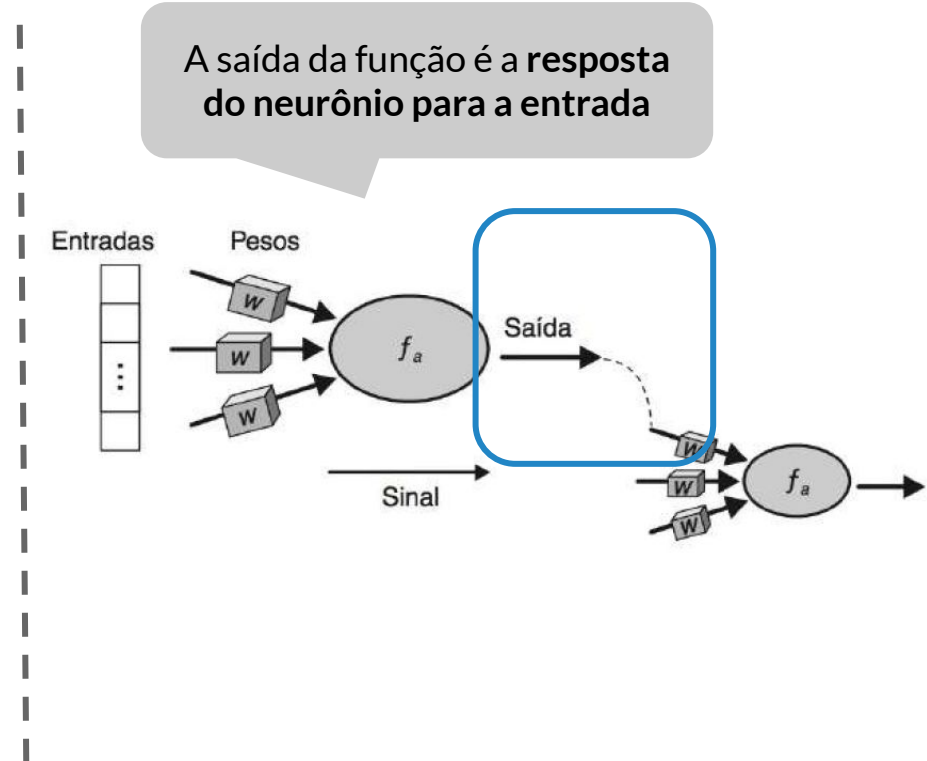
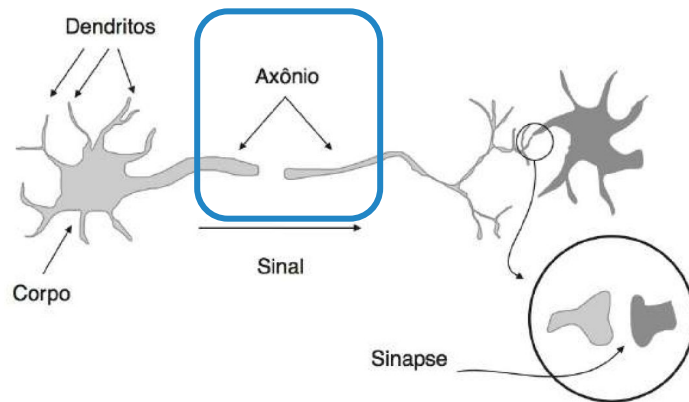


(c) Sigmoidal

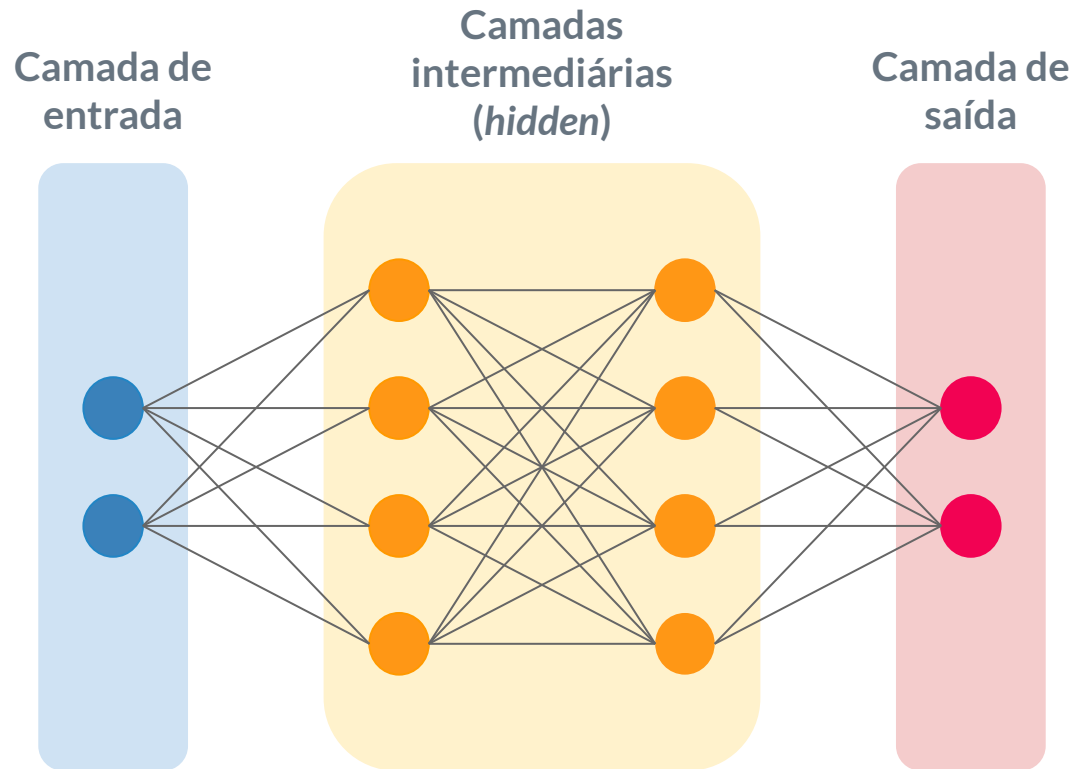
Aproximação contínua e diferenciável da função limiar (podemos usar diferentes inclinações)

# Neurônio Artificial

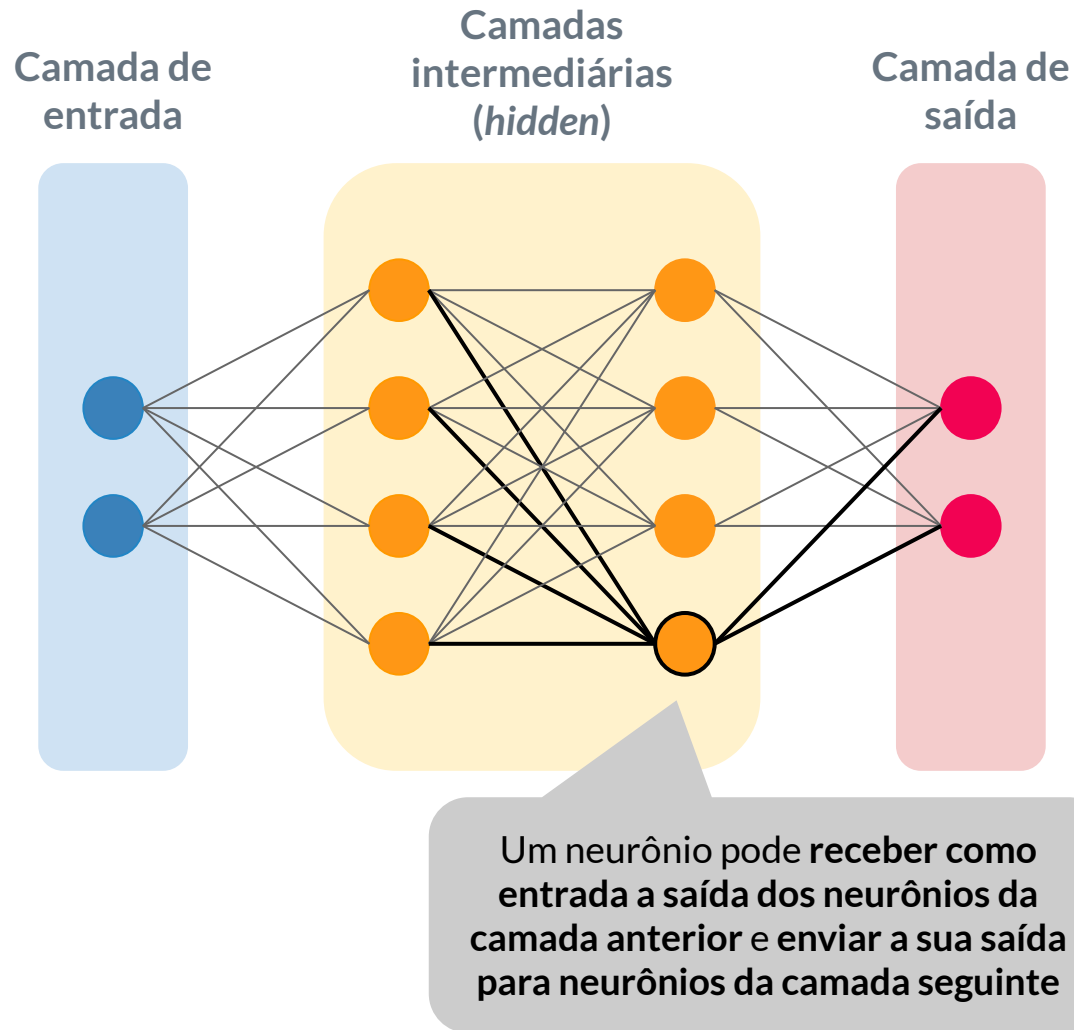
Neurônio → Unidade fundamental de uma RNA



# Redes multicamadas

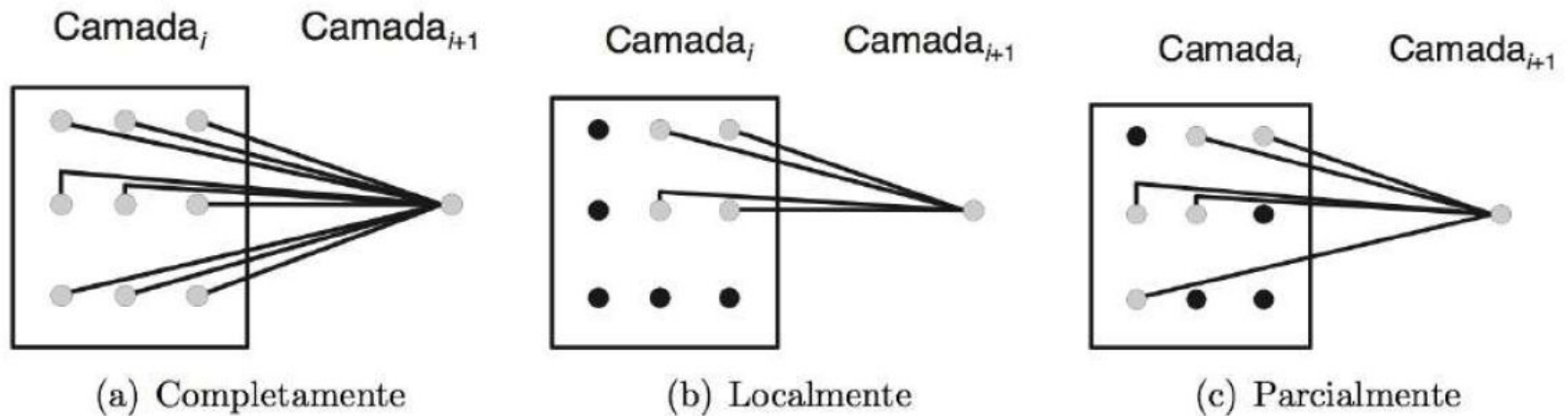


# Redes multicamadas



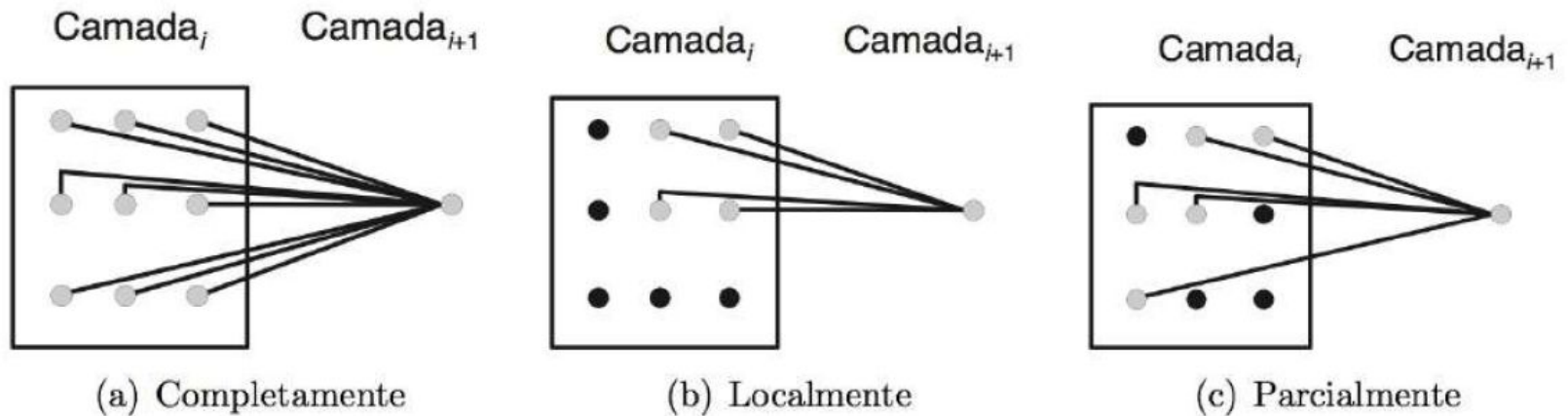
# Redes multicamadas

Diferentes padrões de conexão:



# Redes multicamadas

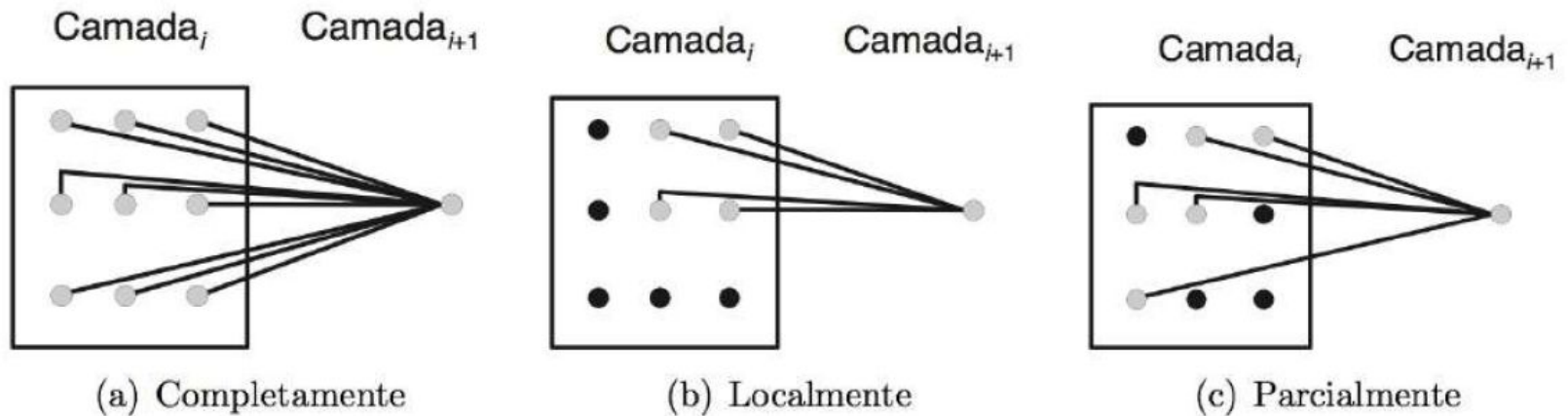
Diferentes padrões de conexão:



Os neurônios da rede estão conectados a **todos** os neurônios da camada anterior e/ou seguinte

# Redes multicamadas

Diferentes padrões de conexão:

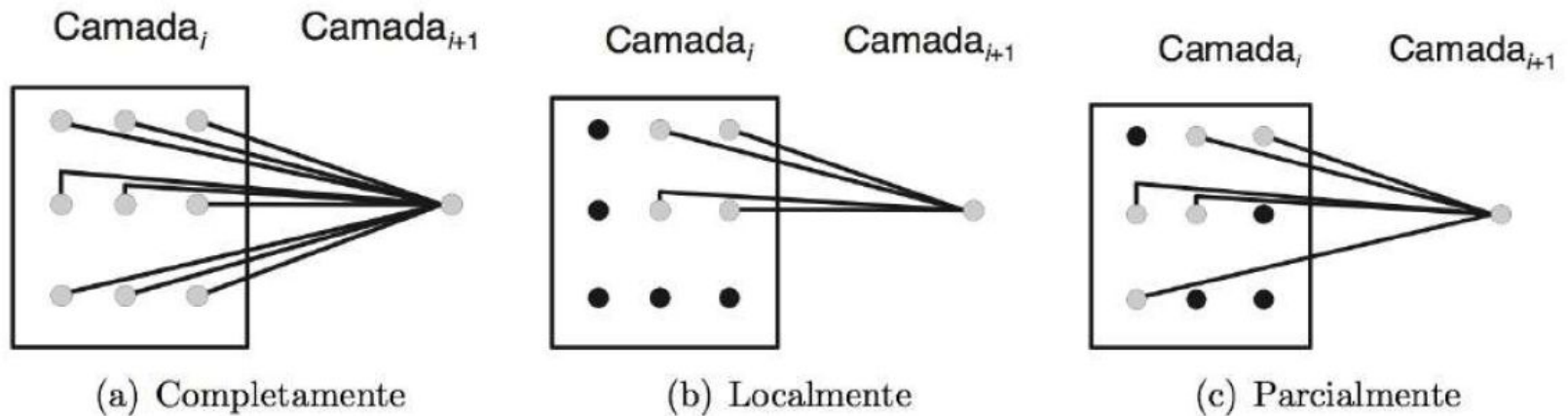


Os neurônios conectados se encontram em uma **região bem definida**



# Redes multicamadas

Diferentes padrões de conexão:



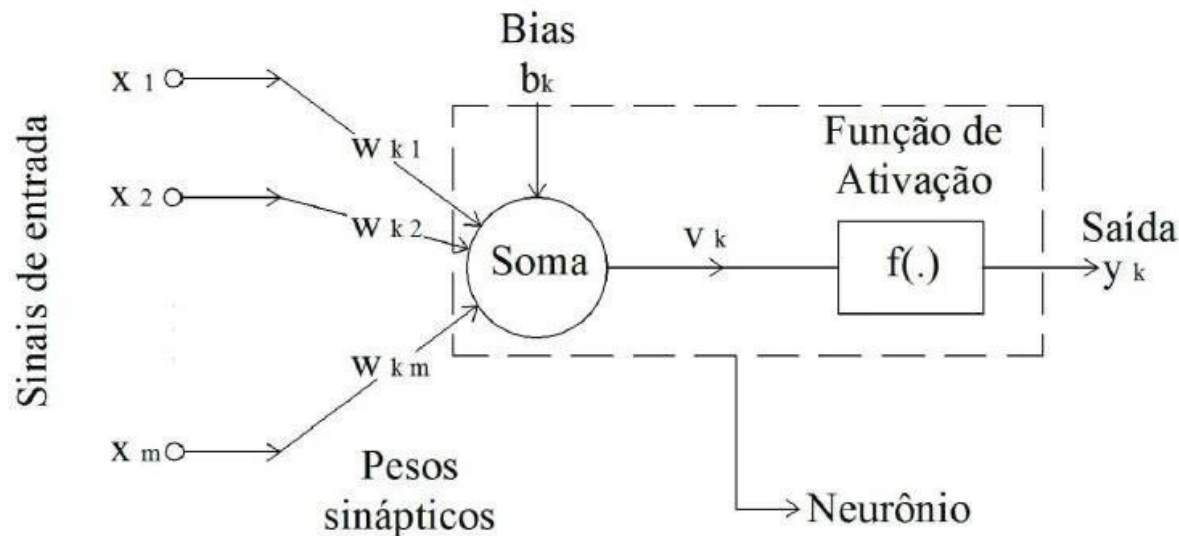
Os neurônios da rede estão conectados apenas a alguns neurônios da camada anterior e/ou seguinte

# 3. Perceptron

*A primeira rede neural a ser implementada*

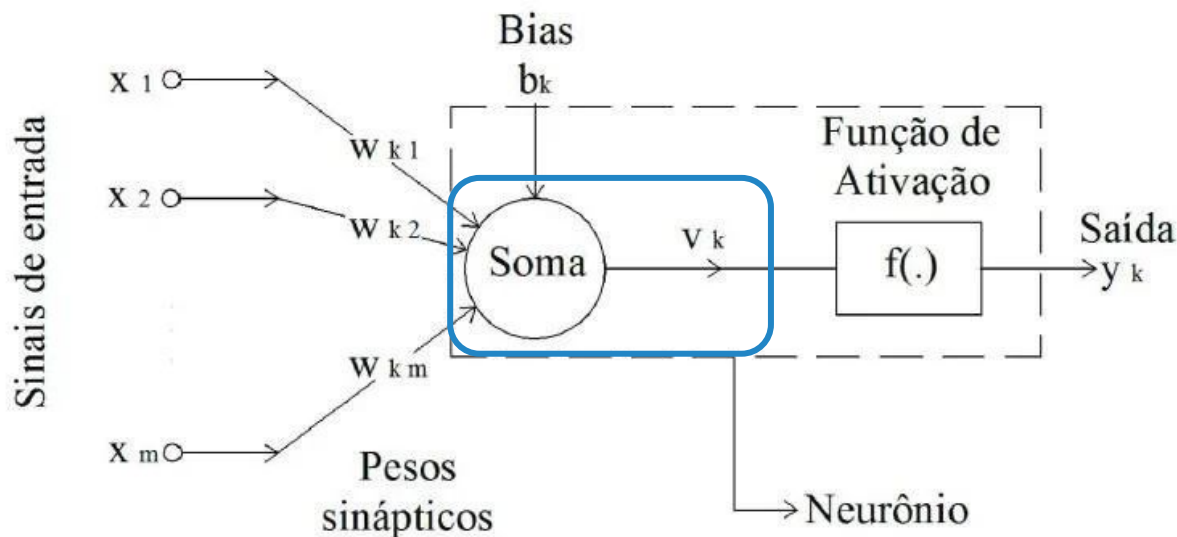
# Neurônio de McCulloch-Pitts

O neurônio de McCulloch-Pitts pode ser modelado como um caso particular de um discriminador linear



# Neurônio de McCulloch-Pitts

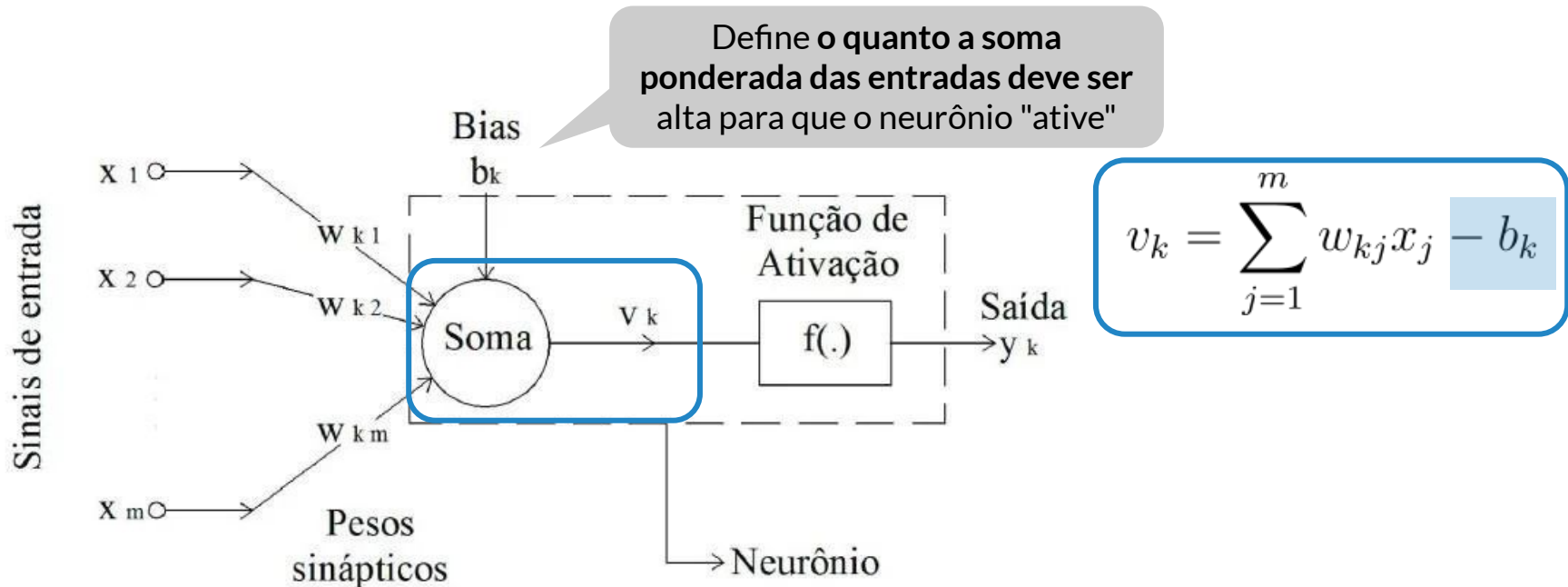
O neurônio de McCulloch-Pitts pode ser modelado como um caso particular de um discriminador linear



$$v_k = \sum_{j=1}^m w_{kj} x_j - b_k$$

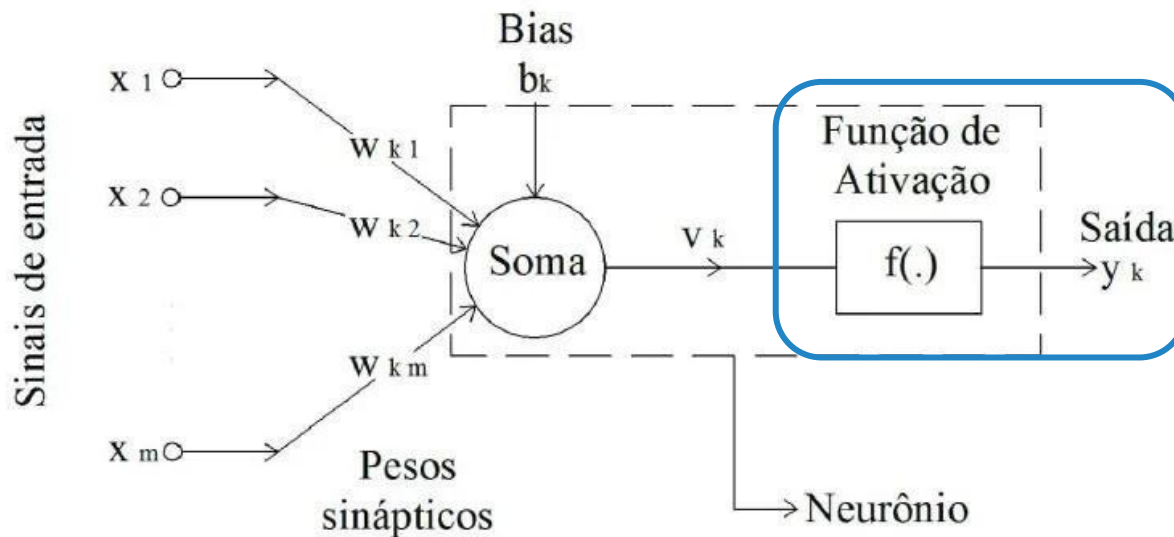
# Neurônio de McCulloch-Pitts

O neurônio de McCulloch-Pitts pode ser modelado como um caso particular de um discriminador linear



# Neurônio de McCulloch-Pitts

O neurônio de McCulloch-Pitts pode ser modelado como um caso particular de um discriminador linear



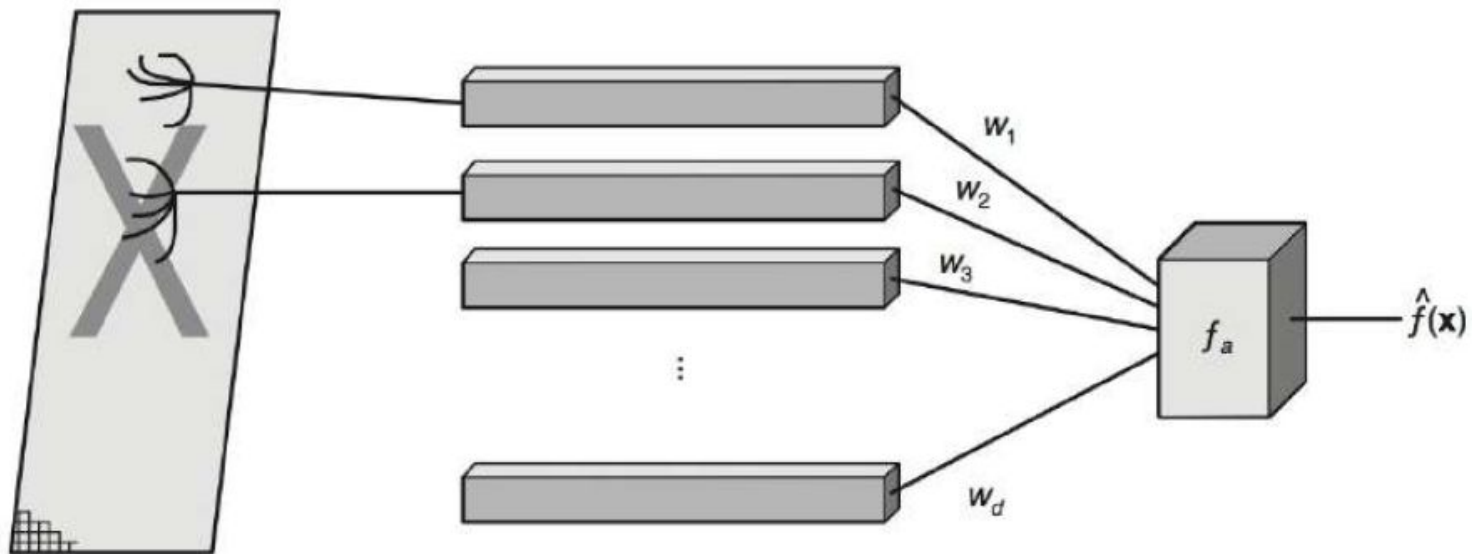
$$v_k = \sum_{j=1}^m w_{kj} x_j - b_k$$

$$y_k = \begin{cases} 1, & \text{se } v_k \geq 0 \\ 0, & \text{se } v_k < 0 \end{cases}$$

Função limiar

# Perceptron

Desenvolvida por Rosenblatt (1958), utiliza o modelo de McCulloch-Pitts com função de limiar e **apenas uma camada de neurônios**



# Perceptron

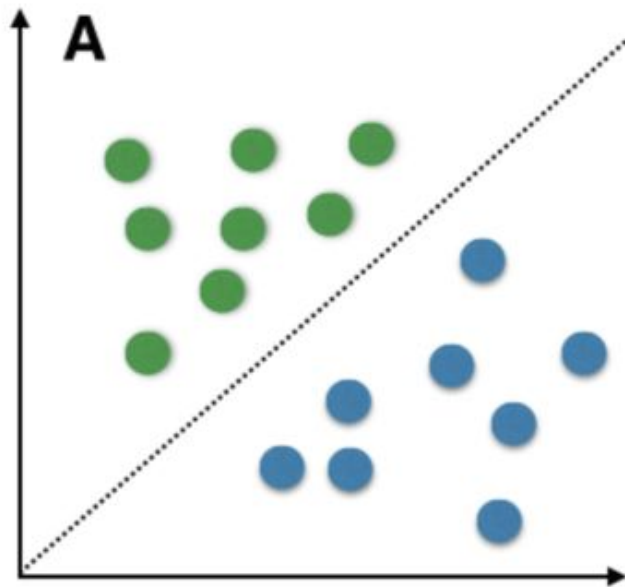
A superfície de decisão (curva de separação) forma um **hiperplano**, ou seja, para um dos lados está uma classe e para o outro lado a outra classe

Podemos entender o Perceptron como uma **superfície de separação em um espaço *n-dimensional*** de instâncias

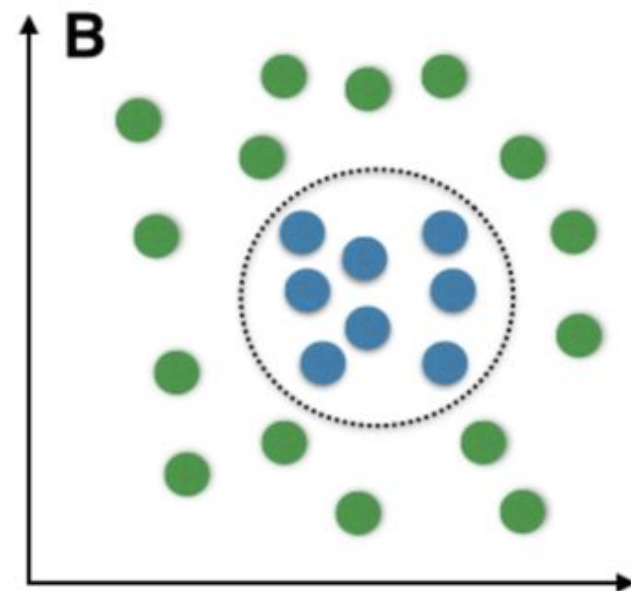
Um único Perceptron consegue separar **somente conjuntos linearmente separáveis**



# Perceptron



Linearmente separáveis



Linearmente não-separáveis

# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

**Algoritmo 7.1** Algoritmo de treinamento da rede perceptron

---

**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio,  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---

# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

## Algoritmo 7.1 Algoritmo de treinamento da rede perceptron

---

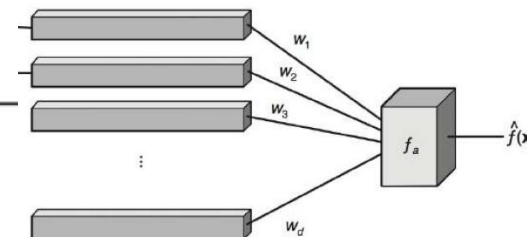
**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio,  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---

Começamos com pesos pequenos (aleatórios ou zero)  
Modelo “nasce sem saber nada”



# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

**Algoritmo 7.1** Algoritmo de treinamento da rede perceptron

---

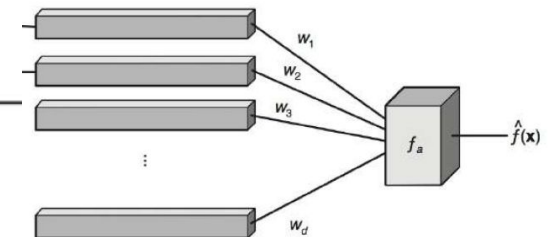
**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio,  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---

Para cada instância do conjunto de treinamento (atributos  $\mathbf{x}_i$  e saída esperada  $y_i$ )



# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

**Algoritmo 7.1** Algoritmo de treinamento da rede perceptron

---

**Entrada:** Um conjunto de  $n$  objetos de treinamento

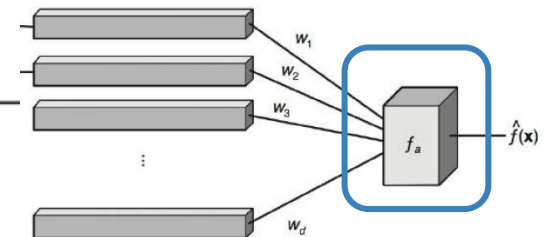
**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio,  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---

- Soma ponderada dos atributos
- Função de ativação (limiar)

$$y_k = \begin{cases} 1, & \text{se } v_k \geq 0 \\ 0, & \text{se } v_k < 0 \end{cases}$$



# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

**Algoritmo 7.1** Algoritmo de treinamento da rede perceptron

---

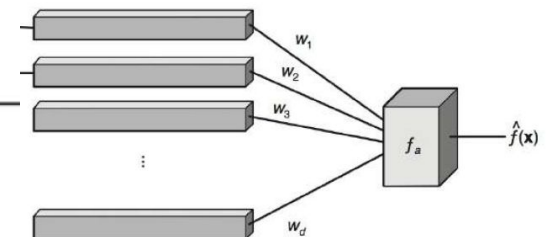
**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---

Erro = esperado - calculado



# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

## Algoritmo 7.1 Algoritmo de treinamento da rede perceptron

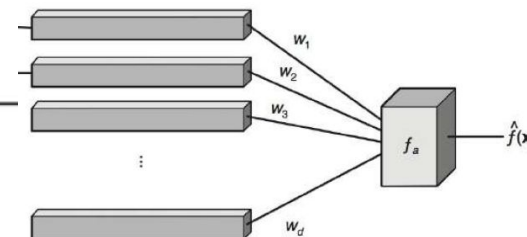
---

**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida por  $\mathbf{x}_i$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

Enquanto o erro > 0 (ou acima de um valor pré-estabelecido), os pesos continuam sendo ajustados





# Perceptron – Treinamento

A ideia é encontrar um conjunto de pesos que defina uma reta que separe as diferentes classes corretamente

---

**Algoritmo 7.1** Algoritmo de treinamento da rede perceptron

---

**Entrada:** Um conjunto de  $n$  objetos de treinamento

**Saída:** Rede perceptron com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores baixos
2 repita
3   para cada objeto  $\mathbf{x}_i$  do conjunto de treinamento faça
4     Calcular valor da saída produzida pelo neurônio,  $\hat{f}(\mathbf{x}_i)$ 
5     Calcular erro =  $y_i - \hat{f}(\mathbf{x}_i)$ 
6     se erro > 0 então
7       Ajustar pesos do neurônio utilizando Equação 7.2
8     fim
9   fim
10 até erro = 0;
```

---



# Perceptron – Treinamento

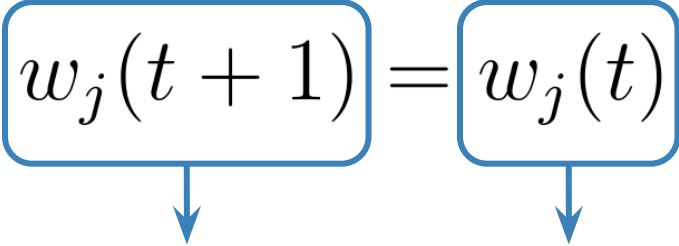
Ajuste de pesos:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$$

# Perceptron – Treinamento

Ajuste de pesos:

$$\boxed{w_j(t+1)} = \boxed{w_j(t)} + \eta x_i^j (y_i - \hat{f}(x_i))$$

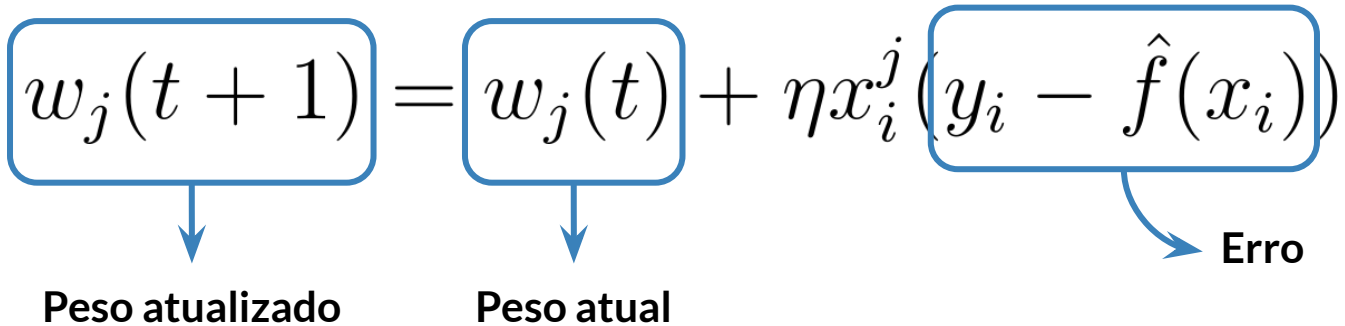


Peso atualizado      Peso atual

# Perceptron – Treinamento

Ajuste de pesos:

$$\boxed{w_j(t+1)} = \boxed{w_j(t)} + \eta x_i^j \boxed{(y_i - \hat{f}(x_i))}$$



Peso atualizado      Peso atual      Erro

# Perceptron – Treinamento

Ajuste de pesos:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$$

The diagram illustrates the weight update equation for a perceptron. The equation is  $w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$ . Each term is enclosed in a blue rounded rectangle. Below  $w_j(t + 1)$  is the label "Peso atualizado" with a downward arrow. Below  $w_j(t)$  is the label "Peso atual" with a downward arrow. Below  $\eta x_i^j$  is the label "Valor do atributo j do exemplo i" with a curved arrow pointing from the term. Below  $(y_i - \hat{f}(x_i))$  is the label "Erro" with a curved arrow pointing from the term.

Peso atualizado

Peso atual

Valor do atributo j do exemplo i

Erro

# Perceptron – Treinamento

Ajuste de pesos:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$$

The diagram illustrates the weight update equation for a perceptron. The equation is  $w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$ . Annotations include: 

- An arrow from  $w_j(t + 1)$  pointing to "Peso atualizado".
- An arrow from  $w_j(t)$  pointing to "Peso atual".
- An arrow from  $\eta$  pointing to a text block defining "Learning rate (taxa de aprendizado)".
- An arrow from  $x_i^j$  pointing to "Valor do atributo j do exemplo i".
- An arrow from  $(y_i - \hat{f}(x_i))$  pointing to "Erro".

**Learning rate (taxa de aprendizado)**  
Magnitude do ajuste feito no peso  
Velocidade de convergência da rede

- Valores altos → grandes variações
- Valores baixos → poucas variações

# Perceptron – Treinamento

Ajuste de pesos:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(x_i))$$

Diagram illustrating the weight update formula for a Perceptron:

- $w_j(t + 1)$ : Peso atualizado (Updated weight)
- $w_j(t)$ : Peso atual (Current weight)
- $\eta$ : **Learning rate (taxa de aprendizado)**  
Magnitude do ajuste feito no peso  
Velocidade de convergência da rede
  - Valores altos → grandes variações
  - Valores baixos → poucas variações
- $x_i^j$ : Valor do atributo j do exemplo i
- $(y_i - \hat{f}(x_i))$ : Erro (Error)

Em resumo:

**Novo peso = peso antigo + (taxa de aprendizado \* entrada \* erro)**

# Perceptron

## Teorema da Convergência da Rede Perceptron

*“Se é possível classificar um conjunto de entradas linearmente, uma rede perceptron fará a classificação.”*

# Perceptron

## Teorema da Convergência da Rede Perceptron

*“Se é possível classificar um conjunto de entradas linearmente, uma rede perceptron fará a classificação.”*

***E se os dados não forem linearmente separáveis?***



# 4. Multilayer Perceptron (MLP)

*Quando adicionamos mais camadas ao Perceptron*

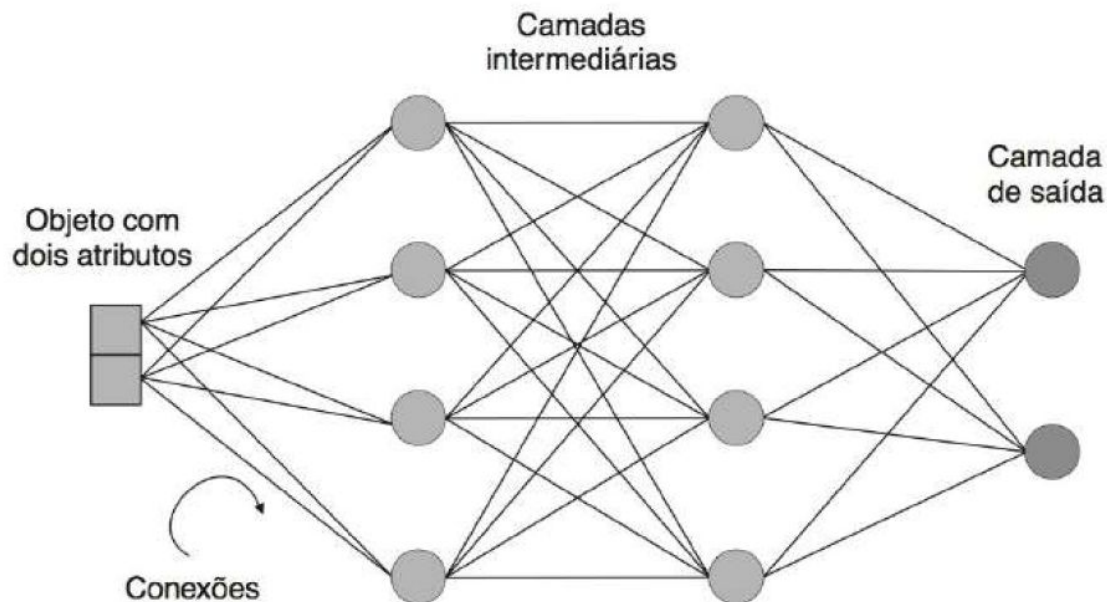
# Multilayer Perceptron (MLP)

Para resolver problemas não linearmente separáveis utilizando RNAs, a alternativa mais utilizada é **adicionar uma ou mais camadas intermediárias**.

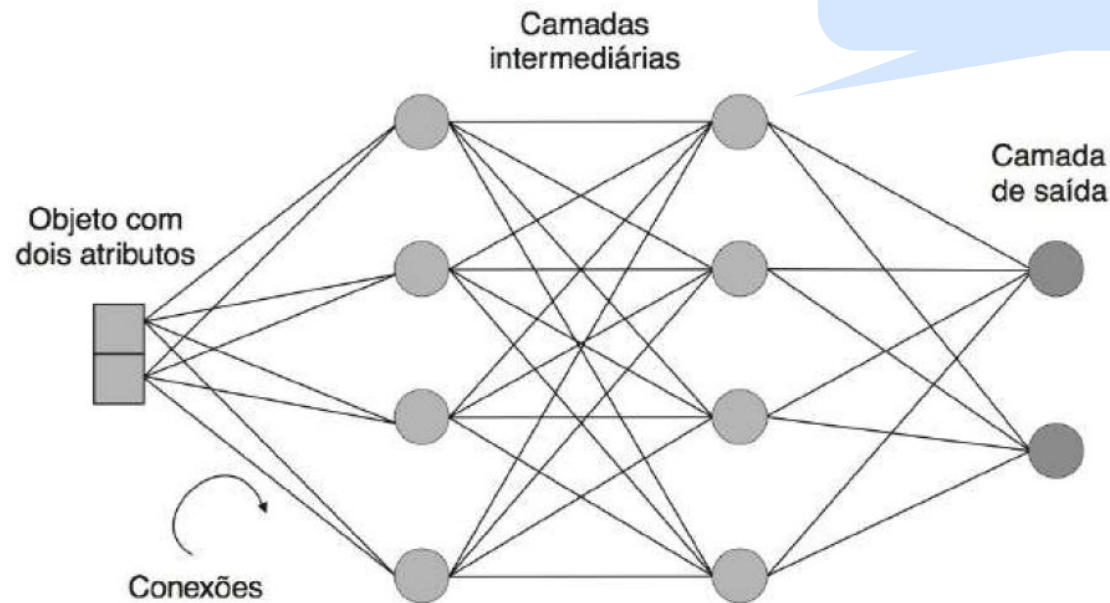
# Multilayer Perceptron (MLP)

**MLP** → uma ou mais camadas intermediárias de neurônios e uma camada de saída

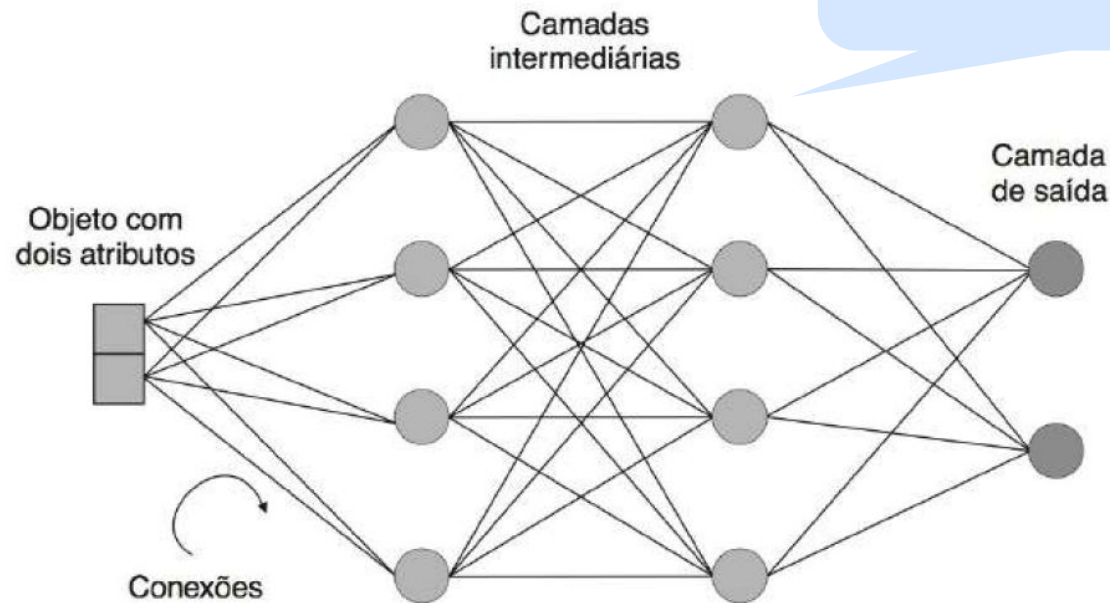
- ▷ Geralmente usam arquitetura **completamente conectada**



# Multilayer Perceptron (MLP)



# Multilayer Perceptron (MLP)

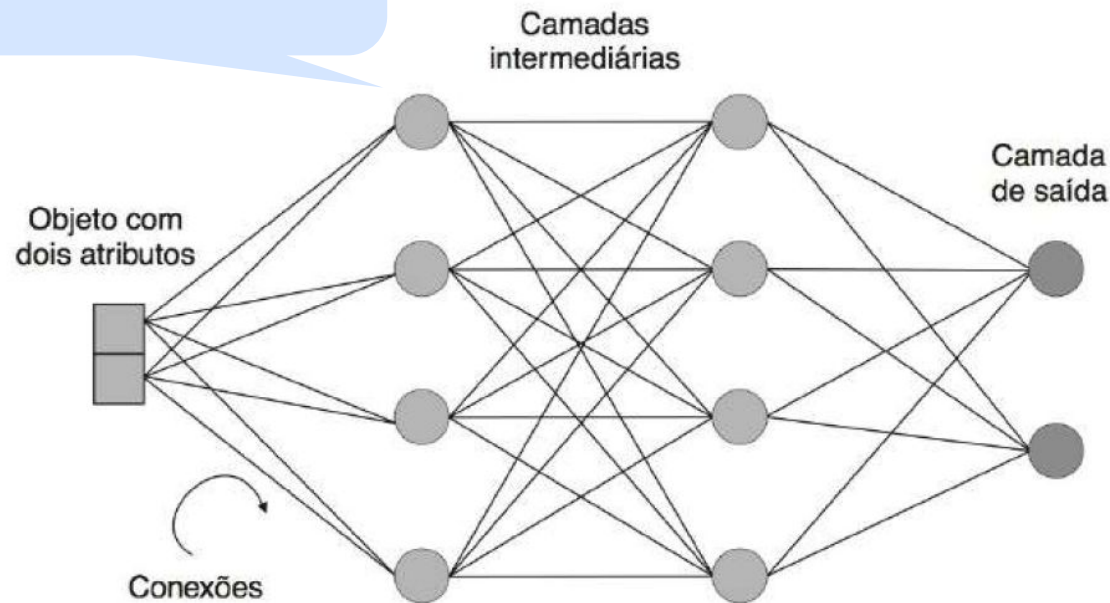


Redes multicamadas usam funções de ativação **não lineares** (i.e., função sigmoideal)

Uma rede multicamadas com funções de ativação lineares é **equivalente a uma rede de uma única camada**

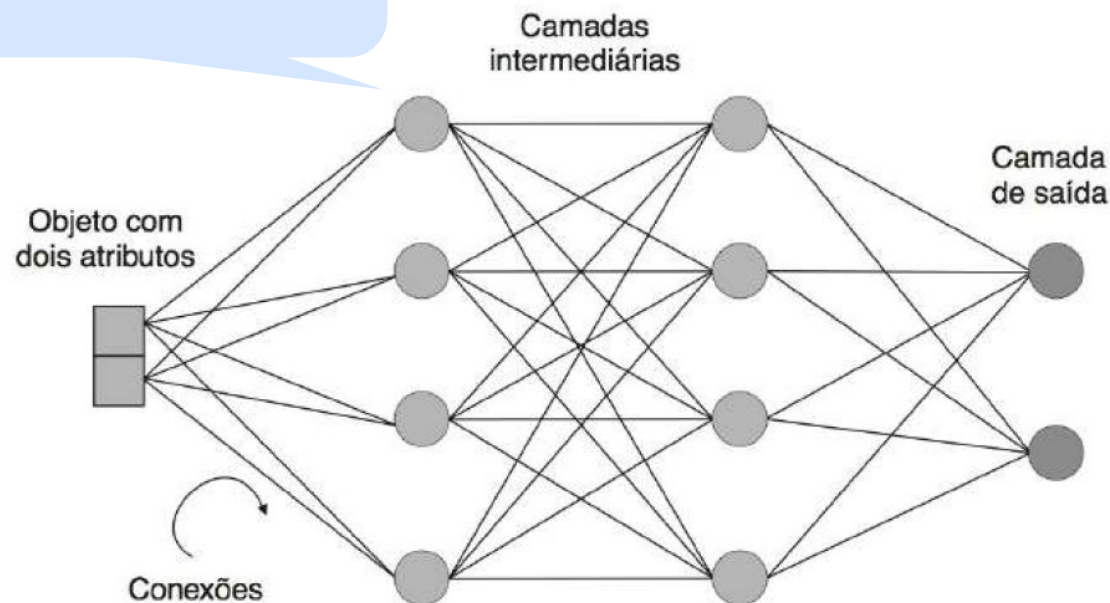
# Multilayer Perceptron (MLP)

Em uma MLP, cada neurônio realiza uma função específica



# Multilayer Perceptron (MLP)

Em uma MLP, cada neurônio realiza uma função específica



À medida em que o processamento avança de uma camada para outra, o processamento se torna mais complexo

# Multilayer Perceptron (MLP)

**Primeira camada** → cada neurônio aprende uma função que define um **hiperplano**

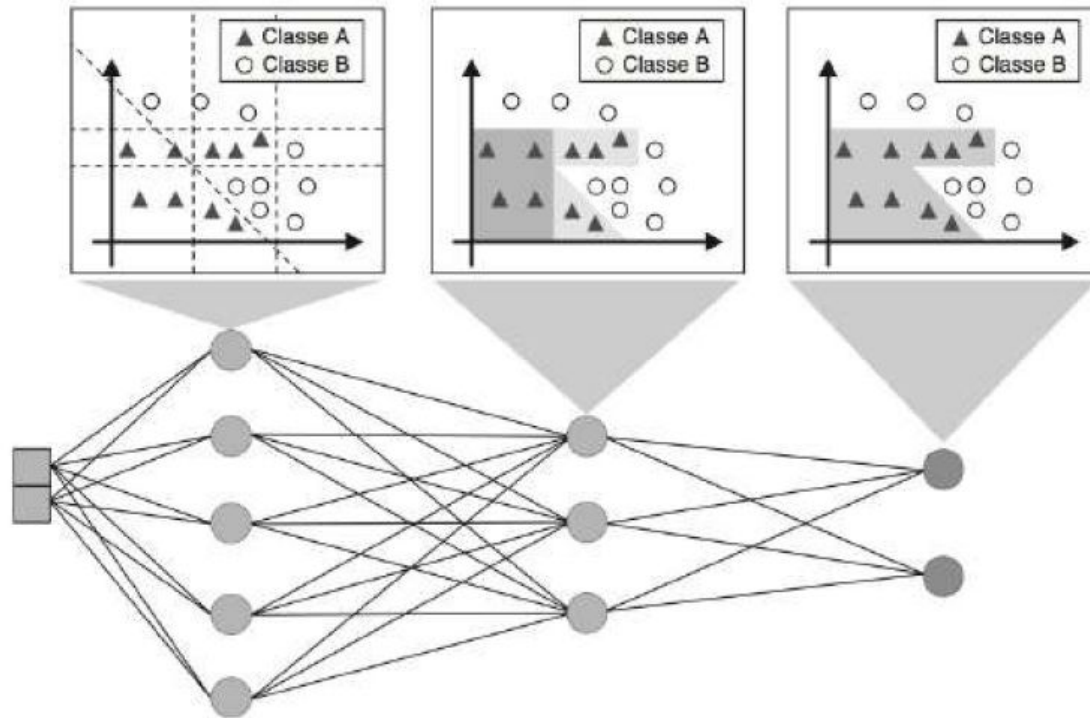
Separa o espaço de entrada em duas partes

**Segunda camada** → combina um grupo de hiperplanos da camada anterior, formando **regiões convexas**

**Camada seguinte** → combinam regiões convexas em regiões de **formato arbitrário**



# Multilayer Perceptron (MLP)



É a combinação das funções desempenhadas por cada neurônio que define a função associada à RNA

# Multilayer Perceptron (MLP)

Cada neurônio da camada de saída está associado a **uma das classes presentes** no conjunto de dados

- ▷ Saída para um objeto de entrada → vetor  $\mathbf{y} = [y_1, y_2, \dots, y_k]^t$ , onde  $k$  é o número de neurônios (e também de classes do problema)
- ▷ **Treinamento** → o vetor de respostas tem o valor 1 na posição associada à classe correta e 0 nas outras
- ▷ **Teste** → classificação é correta quando o valor de saída mais elevado é o do neurônio de saída da classe correta

# Multilayer Perceptron (MLP)

*E como fazemos para treinar redes multicamadas?*

→ Algoritmo backpropagation!

# 5. Exercícios

*Vamos testar os conhecimentos?*

# Exercícios #1

Considere as seguintes afirmações sobre redes neurais artificiais:

- I. Um perceptron elementar só computa funções linearmente separáveis.
- II. Não aceitam valores numéricos como entrada.
- III. O “conhecimento” é representado principalmente através do peso das conexões.

São corretas:

- a) Apenas III
- b) Apenas I e II
- c) Apenas I e III
- d) Apenas II e III
- e) I, II e III

# Referências

FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina**. Rio de Janeiro, RJ: LTC, 2011. (**Capítulo 7**)

Material da Profa. Cristiane Nobre